

Universidad del Valle de Guatemala
Facultad de Ingeniería
Catedrático: Kevin Antonio Velásquez Aguilar
Programación Orientada a Objetos



Ejercicio 4. Modelación con herencia

Nombre: María Isabel Cajas Cuéllar

Carné: 26183

Sección: 20

Fecha de entrega: 28/09/2026

Análisis del Sistema

1.1 Requisitos funcionales del sistema

- Registrar automóviles, motocicletas y camionetas de carga.
- Guardar la placa, marca, modelo y tarifa diaria de cada vehículo.
- Indicar si cada vehículo está disponible o alquilado.
- Registrar la cantidad de pasajeros de los automóviles.
- Registrar si la transmisión de un automóvil es automática o manual.
- Registrar el cilindraje de las motocicletas.
- Registrar la capacidad máxima en toneladas de las camionetas de carga.
- Almacenar los vehículos en una colección dinámica utilizando `ArrayList<Vehiculo>`.
- Buscar un vehículo mediante su placa.
- Mostrar todos los vehículos registrados y sus características.
- Cotizar el alquiler de un vehículo según la cantidad de días.
- Permitir la cotización de vehículos disponibles y alquilados.
- Mostrar el costo total antes de confirmar un alquiler.
- Permitir que el usuario confirme o cancele el alquiler.
- Cambiar el estado del vehículo a alquilado después de confirmar.
- Sumar el costo a los ingresos solamente cuando se confirme el alquiler.
- Impedir que un vehículo tenga dos alquileres activos al mismo tiempo.
- Registrar la devolución de un vehículo alquilado.
- Cambiar el vehículo a disponible después de una devolución.
- Mostrar la cantidad total de vehículos registrados.
- Mostrar cuántos vehículos están disponibles y alquilados.
- Mostrar las cantidades disponibles y alquiladas por categoría.
- Mostrar los ingresos acumulados.
- Mantener el menú funcionando hasta que el usuario seleccione salir.

1.2 Validaciones y restricciones

- La placa, marca y modelo no pueden estar vacíos.
- No se permiten placas repetidas.
- La tarifa diaria debe ser mayor que cero.
- Los pasajeros, el cilindraje y la capacidad de carga deben ser mayores que cero.
- Los días de alquiler deben ser enteros positivos.
- Se controlan entradas numéricas incorrectas con try- catch.
- No se puede alquilar un vehículo ocupado.
- No se puede devolver un vehículo disponible.
- Solo la respuesta S confirma el alquiler; cualquier otra respuesta lo cancela.
- Las cotizaciones y cancelaciones no modifican la disponibilidad ni los ingresos.

- Los ingresos aumentan únicamente al confirmar un alquiler.

1.3 Clases necesarias y su propósito

Clase	Propósito
Vehiculo	Representa la información y los comportamientos comunes de todos los vehículos, como la placa, marca, modelo, tarifa diaria, disponibilidad y cálculo del costo base.
Automovil	Representa un automóvil. Almacena la cantidad de pasajeros y el tipo de transmisión. También calcula el recargo diario cuando es automático.
Motocicleta	Representa una motocicleta. Almacena su cilindraje y calcula el recargo correspondiente cuando supera los 250 cc.
CamionetaCarga	Representa una camioneta de carga. Almacena su capacidad máxima en toneladas y calcula el recargo por capacidad y días.
RentaMovil	Administra la lista de vehículos. Permite registrarlos, buscarlos, mostrarlos, cotizarlos, alquilarlos y devolverlos. También guarda los ingresos acumulados.
App o Main	Ejecuta el programa, crea los vehículos iniciales, muestra el menú, solicita los datos al usuario y llama a los métodos de las demás clases.

1.4 Atributos de cada clase

Clase: Vehiculo

Atributo	Tipo de dato	Visibilidad	Propósito
placa	String	protected	Identifica de manera única al vehículo.

marca	String	protected	Guarda la marca del vehículo.
modelo	String	protected	Guarda el modelo del vehículo.
tarifaDiaria	double	protected	Guarda el precio base del alquiler por día.
disponible	boolean	protected	Indica si el vehículo está disponible o alquilado.

Clase: Automovil

Atributo	Tipo de dato	Visibilidad	Propósito
cantidadPasajeros	int	private	Guarda la cantidad máxima de pasajeros del automóvil.
automatico	boolean	private	Indica si la transmisión es automática o manual.

Clase: Motocicleta

Atributo	Tipo de dato	Visibilidad	Propósito
cilindraje	int	private	Guarda el cilindraje de la motocicleta en centímetros cúbicos.

Clase: CamionetaCarga

Atributo	Tipo de dato	Visibilidad	Propósito
capacidadToneladas	double	private	Guarda la capacidad

			máxima de carga expresada en toneladas.
--	--	--	---

Clase: RentaMovil

Atributo	Tipo de dato	Visibilidad	Propósito
vehiculos	ArrayList<Vehiculo>	private	Almacena dinámicamente todos los vehículos registrados.
ingresos	double	private	Acumula el dinero recibido por los alquileres confirmados.

1.5 Métodos de cada clase

Clase: Vehiculo

Método	Parámetros	Retorno	Visibilidad	Propósito
Vehiculo	String placa, String marca, String modelo, double tarifaDiaria	No aplica	public	Constructor que asigna los datos comunes y coloca el vehículo como disponible.
calcularCostoAlquiler	int dias	double	public	Calcula el costo base multiplicando la tarifa diaria por los días.
alquilar	Ninguno	void	public	Cambia la disponibilidad

				del vehículo a false.
devolver	Ninguno	void	public	Cambia la disponibilidad del vehículo a true.
getPlaca	Ninguno	String	public	Devuelve la placa del vehículo.
getMarca	Ninguno	String	public	Devuelve la marca del vehículo.
getModelo	Ninguno	String	public	Devuelve el modelo del vehículo.
getTarifaDiaria	Ninguno	double	public	Devuelve la tarifa diaria.
isDisponible	Ninguno	boolean	public	Devuelve true si el vehículo está disponible y false si está alquilado.

Clase Automovil

Método	Parámetros	Retorno	Visibilidad	Propósito
Automovil	Datos comunes, int cantidadPasajeros, boolean automatico	No aplica	public	Constructor que utiliza super para inicializar los datos heredados y asigna los

				datos particulares.
calcularCostoAlquiler	int dias	double	public @Override	Calcula el costo base y agrega Q50 por cada día si el automóvil es automático.
getCantidadPasajeros	Ninguno	int	public	Devuelve la cantidad de pasajeros.
isAutomatico	Ninguno	boolean	public	Indica si la transmisión es automática.

Clase: Motocicleta

Método	Parámetros	Retorno	Visibilidad	Propósito
Motocicleta	Datos comunes, int cilindraje	No aplica	public	Constructor que utiliza super e inicializa el cilindraje.
calcularCostoAlquiler	int dias	double	public @Override	Calcula el costo base y agrega Q75 una vez si el cilindraje supera los 250 cc.
getCilindraje	Ninguno	int	public	Devuelve el cilindraje de la motocicleta.

Clase: CamionetaCarga

Método	Parámetros	Retorno	Visibilidad	Propósito
CamionetaCarga	Datos comunes, double capacidadToneladas	No aplica	public	Constructor que utiliza super e inicializa la capacidad de carga.
calcularCostoAlquiler	int dias	double	public @Override	Calcula el costo base y agrega Q100 por tonelada y por día.
getCapacidadToneladas	Ninguno	double	public	Devuelve la capacidad máxima de carga.

Clase: RentaMovil

Método	Parámetros	Retorno	Visibilidad	Propósito
RentaMovil	Ninguno	No aplica	public	Constructor que inicializa la lista de vehículos y establece los ingresos en cero.
registrarVehiculo	Vehiculo vehiculo	void	public	Agrega un vehículo a la lista después de verificar que su placa no esté repetida.

buscarVehiculo	String placa	Vehiculo	public	Recorre la lista y devuelve el vehículo que tenga la placa indicada. Si no existe, devuelve null.
mostrarVehiculos	Ninguno	void	public	Muestra los datos y la disponibilidad de todos los vehículos registrados.
cotizar	String placa, int dias	void	public	Busca el vehículo, calcula el costo y muestra la cotización sin modificar su estado ni los ingresos.
alquilarVehiculo	String placa, int dias	void	public	Verifica que el vehículo esté disponible, calcula el costo, registra el alquiler y suma el total a los ingresos.
devolverVehiculo	String placa	void	public	Verifica que el vehículo esté alquilado y lo cambia nuevamente a disponible.
getIngresos	Ninguno	double	public	Devuelve los ingresos acumulados.
getVehiculos	Ninguno	ArrayList<Vehiculo>	public	Devuelve la lista de vehículos registrados.

Clase: App o Main

Método	Parámetros	Retorno	Visibilidad	Propósito
main	String[] args	void	public static	Inicia el programa, crea los vehículos iniciales, muestra el menú y dirige la ejecución.
registrarVehiculo	Scanner scanner, RentaMovil rentaMovil	void	public static	Solicita los datos y crea el tipo de vehículo seleccionado.
cotizar	Scanner scanner, RentaMovil rentaMovil	void	public static	Solicita la placa y los días para realizar una cotización.
alquilar	Scanner scanner, RentaMovil rentaMovil	void	public static	Comprueba la disponibilidad, muestra el costo y solicita la confirmación.
devolver	Scanner scanner, RentaMovil rentaMovil	void	public static	Solicita la placa y registra la devolución del vehículo.
mostrarReporte	RentaMovil rentaMovil	void	public static	Cuenta los vehículos por categoría y disponibilidad, y muestra los ingresos.
leerEntero	Scanner scanner,	int	public static	Lee un número entero y controla los

	String mensaje			errores de formato mediante try-catch.
leerEnteroPositivo	Scanner scanner, String mensaje	int	public static	Solicita un número entero hasta que sea mayor que cero.
leerDoublePositivo	Scanner scanner, String mensaje	double	public static	Solicita un número decimal hasta que sea mayor que cero.

1.6 Valores iniciales de los objetos

- Los objetos reciben sus valores iniciales mediante sus constructores. El constructor de “Vehiculo” recibe la placa, marca, modelo y tarifa diaria, y establece automáticamente la disponibilidad en “true”, porque todo vehículo nuevo comienza disponible.
- Los constructores de “Automovil”, “Motocicleta” y “CamionetaCarga” utilizan “super” para inicializar los datos heredados de “Vehiculo”. Cada subclase asigna sus características particulares, como la cantidad de pasajeros, el cilindraje o la capacidad de carga.
- El constructor de “RentaMovil” crea un “Arraylist” vacío para almacenar los vehículos y establece los ingresos iniciales en cero. La clase “Main” registra los vehículos iniciales que se utilizarán para probar el programa.

1.7 Organización de las características comunes y particulares

Se utiliza herencia porque “Automovil”, “Motocicleta” y “CamionetaCarga” son tipos de “Vehiculo”.

La clase “Vehiculo” contiene la información que todas las categorías comparten: “Placa”, “Marca”, “Modelo”, “Tarifa diaria”, “Disponibilidad”, “Cálculo del costo base”, “Operaciones de alquiler” y “devolución”.

Características particulares de subclases

Subclase	Características particulares
Automovil	Cantidad de pasajeros y tipo de transmisión.
Motocicleta	Cilindraje.
CamionetaCarga	Capacidad máxima en toneladas.

Esta organización evita repetir los mismos atributos y métodos en cada categoría. Además, permite almacenar todos los objetos en un solo `ArrayList<Vehiculo>`, aunque pertenezcan a diferentes subclases.

Cada subclase utiliza `@Override` para modificar el método `calcularCostoAlquiler(int dias)` y aplicar su propia regla de cobro. De esta manera, cada vehículo es responsable de calcular correctamente el precio de su alquiler.

1.8 Decisiones de cobro y colaboración entre los objetos

La clase `Vehiculo` realiza el cálculo general del alquiler, multiplicando la tarifa diaria por la cantidad de días: $\text{tarifaDiaria} * \text{días}$

Cada subclase sobrescribe el método `calcularCostoAlquiler` para aplicar su regla particular:

- `Automovil` agrega Q50 por cada día cuando la transmisión es automática.
- `Motocicleta` agrega Q75 una sola vez cuando el cilindraje supera los 250 cc.
- `CamionetaCarga` agrega Q100 por cada tonelada de capacidad y por cada día.

Para completar un alquiler, los objetos colaboran de la siguiente manera:

1. `Main` solicita la placa y la cantidad de días.
2. `RentaMovil` busca el vehículo mediante su placa.
3. El objeto encontrado ejecuta `calcularCostoAlquiler(dias)`.
4. Java utiliza automáticamente el `@Override` de la categoría correspondiente.
5. `Main` muestra el costo total y solicita la confirmación.
6. Si el usuario confirma, `RentaMovil` cambia el vehículo a ocupado y suma el costo a los ingresos.
7. Si el usuario no confirma, el alquiler se cancela y la información permanece igual.

De esta manera, la “`Main`” controla la interacción con el usuario, “`RentaMovil`” administra la operación y cada vehículo calcula su propio costo.

1.9 Incorporación de una nueva categoría

- Para incorporar otra categoría, se crea una nueva clase que herede de “`Vehiculo`”. Esta clase incluiría sus atributos particulares y sobrescribiría `calcularCostoAlquiler(int dias)`.
- No sería necesario modificar las categorías existentes. Solamente se agregaría la nueva opción de registro en “`Main`” y su conteo en el reporte

Diagrama UML

